Lesson 1 · What Computer Graphics Is, and Your First Image

Course: Graphics with C++ | Level: Absolute beginner | Est. 45–60 min

Welcome. This is the first lesson of a course we'll grow together, one Markdown file (and one PDF) at a time. By the end of *this* lesson you will have written a real C++ program that produces a real image file — a color gradient — completely from scratch, using **no graphics libraries at all**. That's the honest way to learn graphics: understand the pixels before you let a library hide them from you.

We'll also stop and explain every piece of jargon as it appears. Nothing is assumed.

---

1. What "computer graphics" actually means

Computer graphics is the craft of **turning numbers into pictures**. Somewhere in your computer there is a grid of tiny colored squares (your screen). Graphics is the set of techniques for deciding what color each of those squares should be.

> **THE ONE IDEA TO HOLD ONTO**
>
> An image is just a grid of numbers. A program that "does graphics" is a program that fills in those numbers. Everything else — 3D, lighting, games, movies — is an elaborate way of computing those numbers.

The word for "compute the final colors of an image" is **rendering**.

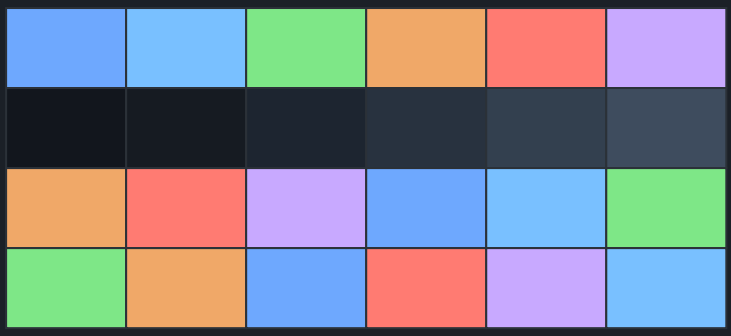
> **JARGON · RENDERING**
>
> **Rendering** is the process of producing an image from a description of a scene. The description might be "a red triangle at these coordinates" or "a spaceship lit by two lights." The renderer's job is to turn that description into concrete pixel colors.

2. Pixels and the framebuffer

Zoom far enough into any screen and you find **pixels** — the smallest addressable dots of color.

> **JARGON · PIXEL**
>
> **Pixel** is short for "picture element." It is one dot in the image grid, and it stores a single color.

A 6 × 4 image = 24 pixels, each holding one color



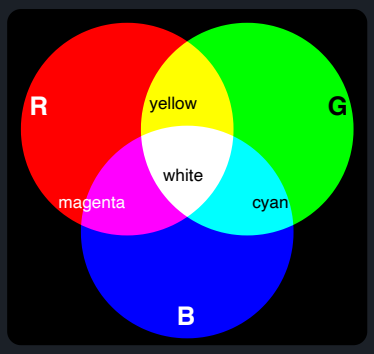
Every image is a rectangular grid of pixels. Width × height = total pixel count.

While your program is computing an image, it keeps those pixel colors somewhere in memory. That block of memory is called the **framebuffer**.

> **JARGON · FRAMEBUFFER**
>
> A **framebuffer** is the region of memory that holds the color of every pixel for one image ("frame"). Rendering = writing values into the framebuffer. Displaying = copying the framebuffer to the screen.

3. How a color is stored: RGB

Each pixel's color is almost always described with three numbers: how much **R**ed, **G**reen, and **B**lue light it emits. Mix those three and you can make (nearly) any color the eye perceives. This is the **RGB color model**.



Additive light: R + G = yellow, R + B = magenta, G + B = cyan, and all three together = white. Zero of all three is black. (Rendered with real additive blending, so the overlaps are accurate.)

Each of the three channels is usually a whole number from **0 to 255**. Why 255? Because one channel is stored in a single **byte** (8 bits), and 8 bits can represent 256 distinct values, 0 through 255.

> **JARGON · BIT AND BYTE**
>
> A **bit** is a single 0-or-1. A **byte** is 8 bits grouped together, giving $2^8 = 256$ possible values (0–255). One byte per color channel is the classic "24-bit color" you've seen in image settings (8 + 8 + 8).

Examples: `(255, 0, 0)` is pure red, `(0, 0, 0)` is black, `(255, 255, 255)` is white, `(255, 165, 0)` is orange.

4. Where is pixel (0,0)? The image coordinate system

To fill in pixels we need to *address* them with coordinates. In image code the convention is: **x grows to the right, y grows downward, and (0,0) is the top-left corner.**

> **WATCH OUT**
>
> In school math, y goes *up*. In image and screen coordinates, y goes *down*. This trips up everyone at first. We'll be explicit about it every time it matters.

Image coordinates: origin at top-left, y increases downward.

5. Setting up C++ (and the jargon that comes with it)

You already have a **compiler** installed (Apple's `clang++`). Let's define the words you'll hear constantly.

> **JARGON · THE C++ TOOLCHAIN**
>
> **Source file** — a text file of C++ code you write, ending in `.cpp`.
> **Compiler** — a program that translates your source file into machine instructions the CPU can run.
> **Compiling** — the act of running the compiler on your source file.
> **Executable / binary** — the runnable program the compiler produces.
> **Header** — a file (often `<iostream>`) that declares functionality you want to *use*. You "include" it to gain access.

6. The plan: write an image with zero libraries

We'll output the **PPM** image format. It's the simplest real image format in existence — it's basically plain text listing the pixels. No library needed; we just print numbers. Later we'll open it with any image viewer (or convert it to PNG).

JARGON · PPM FORMAT

PPM (Portable Pixmap) is a trivially simple image format. A small text header says "I'm an image, this wide, this tall, colors go up to 255," followed by the R G B values of every pixel, row by row, top to bottom.

A PPM file looks like this for a tiny 2×2 image:

```
P3
2 2
255
255 0 0    0 255 0
0 0 255    255 255 255
```

That header means: `P3` = "ASCII RGB PPM"; `2 2` = width and height; `255` = the maximum value a channel can hold. Then four pixels: red, green, blue, white.

7. Your first program: a color gradient

Here is the complete program. Read it once top to bottom, then we'll dissect it.

```cpp
#include <iostream>   // gives us std::cout for printing

int main() {
    const int width  = 256;
    const int height = 256;

    // PPM header: format, dimensions, max color value.
    std::cout << "P3\n" << width << ' ' << height << "\n255\n";

    // Fill the image top-to-bottom (y), left-to-right (x).
    for (int y = 0; y < height; ++y) {
        for (int x = 0; x < width; ++x) {
            int r = x;                       // red grows to the right
            int g = y;                       // green grows downward
            int b = 128;                     // constant blue

            std::cout << r << ' ' << g << ' ' << b << '\n';
        }
    }

    return 0;
}
```

Dissecting it, line by line

- `#include <iostream>` — pull in the **header** that lets us print text. `iostream` = "input/output stream."
- `int main() { ... }` — every C++ program starts running at a **function** named `main`. The `int` says `main` hands back a whole number when it finishes.

> **JARGON · FUNCTION**
>
> A **function** is a named, reusable block of code. `main` is special: it's the entry point the operating system calls to start your program. The `{ }` curly braces enclose the function's body.

- `const int width = 256;` — declare a **variable** named `width` holding the whole number 256. `int` is its **type** (integer). `const` means "this never changes."

> **JARGON · VARIABLE AND TYPE**
>
> A **variable** is a named box in memory holding a value. Its **type** (like `int` for integers) tells the compiler what kind of value it holds and how big the box is.

- `std::cout << ...` — `std::cout` is the **standard output stream** (your terminal). The `<<` operator sends things into it. We redirect that output into a file when we run the program, so everything printed *becomes the image file*.

> **JARGON · STD, NAMESPACE, AND ::**
>
> `std` is the **standard library**'s **namespace** — a labeled container of names so the library's `cout` doesn't collide with any `cout` you might define. The `::` means "reach inside this namespace," so `std::cout` = "the `cout` that lives in `std`."

- The two `for` loops are the heart of it. The outer loop walks every **row** `y` from top (0) to bottom. The inner loop walks every **column** `x` left to right. Together they visit all 256 × 256 = 65,536 pixels exactly once.

> **JARGON · FOR LOOP**
>
> A **for loop** repeats a block of code. `for (int x = 0; x < width; ++x)` reads as: start with `x = 0`; keep going while `x < width`; after each pass do `++x` (add one to `x`). Nesting one loop inside another is how we sweep a 2D grid.

- Inside, we choose this pixel's color: red equals the column, green equals the row, blue is a flat 128. So the left edge is dark red, the right edge is bright red, the top is dark green, the bottom is bright green — a smooth two-way gradient.
- `return 0;` — tell the operating system "the program finished successfully." By convention, `0` means success.

8. How the code becomes an image

When this program is compiled into a runnable form and executed, everything it prints with `std::cout` can be captured into a file instead of shown on screen. Because what it prints is exactly the PPM format from §6, that file *is* an image — conventionally named `gradient.ppm`.

JARGON · COMPILING, RUNNING, AND REDIRECTING

Compiling turns the `.cpp` source into an executable program (the language version and optimization level are chosen with compiler options like `-std=c++17` and `-O2`). **Running** it makes it emit the pixel numbers. A shell **redirect** — the `>` symbol — sends that printed output into a file instead of the terminal, which is what saves the image. A `.ppm` file can then be turned into the more common `.png` with macOS's built-in `sips` tool.

Compiled and run, the program produces a square that fades from dark in the top-left toward bright red on the right and bright green toward the bottom:

Your rendered gradient.png (blue held at 128): dark blue (0,0,128) top-left, magenta (255,0,128) top-right, spring green (0,255,128) bottom-left, pale yellow (255,255,128) bottom-right.

YOU JUST DID REAL GRAPHICS

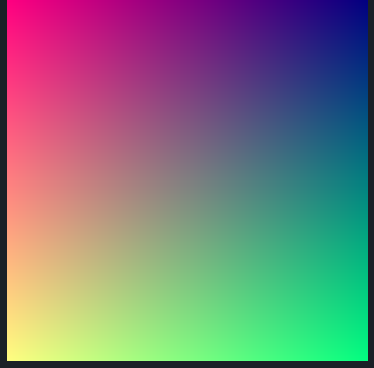
No engine, no library — you decided the color of 65,536 pixels with a formula and wrote them to disk. That loop-over-every-pixel-and-compute-a-color is the exact shape of a **ray tracer**, which we'll build in a few lessons. The only thing that changes is the formula inside the loop.

9. More worked examples

The whole image is decided by the three lines that set `r`, `g`, `b`. Change only those and you get a completely different picture — the loops never change. Here's what a few formulas produce:

A · Flip a channel

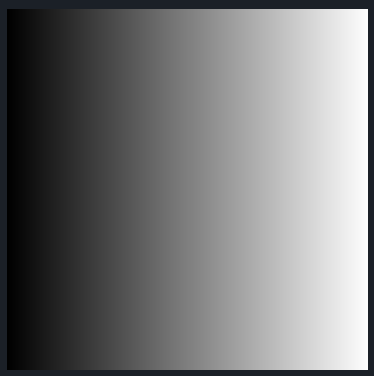
Replace `int r = x;` with `int r = 255 - x;`. Now red is brightest where `x = 0`, so the **bright red edge moves to the left** instead of the right. Subtracting a coordinate from 255 mirrors that channel.



`r = 255 - x`: *the bright-red edge is now on the left — the whole picture is the original gradient mirrored left-to-right.*

B · Grayscale

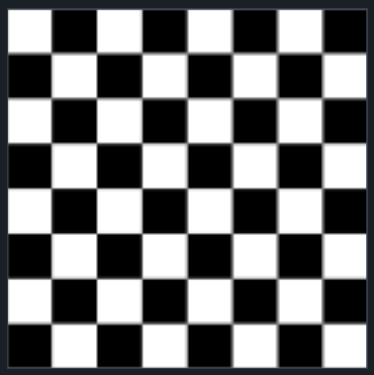
Set all three channels to the same value: `int r = x, g = x, b = x;`. Whenever R, G, and B are equal the pixel is a shade of **gray** — no channel dominates. Since the value here depends only on `x`, the result is vertical bands fading from black on the left (`x = 0`) to white on the right (`x = 255`).



`r = g = b = x`: *equal channels look gray, and depending only on* `x` *gives a smooth black-to-white ramp across the columns.*

C · Checkerboard

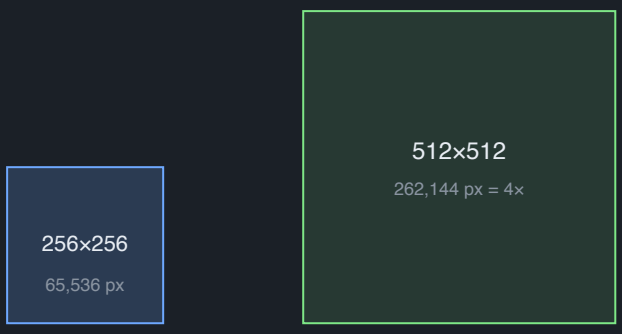
Make the color depend on which 32-pixel block a pixel lands in: white when `(x / 32 + y / 32)` is even, black when odd. Because `x / 32` is integer division, each 32×32 block shares one value, and adding the two block indices flips between even and odd across the grid — a **checkerboard**. (`n % 2 == 0` tests "even", using the remainder operator `%`.)



White when `(x / 32 + y / 32)` *is even: a checkerboard. The top-left block is white because 0 + 0 is even. A 256-wide image has 8 blocks of 32 pixels across, so an 8×8 board.*

D · A bigger image

Change `width` and `height` from 256 to 512. That doubles each side, so there are **4× as many pixels** (512 × 512 = 262,144, versus 65,536) — and the output file grows about 4× too, since every pixel writes its own line of numbers.



Doubling each side quadruples the area: the 512×512 image holds four times the pixels (and takes ~4× the memory and file size) of the 256×256 one.

10. What you learned

- An image is a **grid of pixels**; each pixel is an **RGB** triple of bytes (0–255).
- The in-memory grid you write into is the **framebuffer**.
- Image coordinates put **(0,0) at the top-left**, with **y pointing down**.
- You can produce a real image with nothing but `std::cout` and the **PPM** format.
- The core loop of graphics is "*for every pixel, compute a color.*"
- C++ words now in your vocabulary: compiler, source file, executable, header, `#include`, function, `main`, variable, type, `int`, `const`, namespace, `std::cout`, `for` loop, shell redirect.

> When something is unclear or you want a concept expanded, tell me the section and I'll revise this lesson and regenerate the PDF — that's the whole point of keeping it as living Markdown.